
FILE: PROJECT OPEN

CLASSIFIED TRAINING RUN / LOCAL DEFENSIVE AUDIT

FUZZING VS AI REVIEW

NIM ASYNCHTTPSERVER

/usr/local/audit

READY

audit> run comparison -target std/asynchttpserver

audit> require evidence -payload -reproducer -replay

FILE: QUESTION

Which tool produced evidence that survived replay?

METHOD A

FUZZING

execute malformed inputs until behavior changes

METHOD B

AI REVIEW

inspect source and propose risks to validate

Evaluation standard: exact input, direct reproducer, fixed-version replay.

FILE: RESULT SUMMARY

1

confirmed runtime crash

0

AI-confirmed crashes

HTTP/

minimal crashing protocol

/audit/result

READY

audit> fuzzer: saved crashing input

audit> reviewer: reproduced crash locally

audit> fixed build: replay no longer crashes

FILE: ATTACK SURFACE

The boundary is plain text
from an unauthenticated
client.

request line	attacker controlled
headers	attacker controlled
body	attacker controlled

Scope: local defensive testing against an intentionally older
Nim version.

/net/input

READY

```
GET /index.html HTTP/1.1
Host: example.local
Content-Length: 12
```

```
optional-body
```

risk: malformed bytes reach parser state

FILE: FUZZING LOOP

Coverage guidance turns random input into a search process.



/build/fuzzer

READY

```
nim c --cc:clang -d:useMalloc --noMain:on \  
  --passC:-fsanitize=fuzzer,address,undefined \  
  --passL:-fsanitize=fuzzer,address,undefined \  
  asynchttpserver_fuzzer.nim
```

FILE: CRASH INPUT

MINIMAL REPRODUCER

HTTP/

An incomplete protocol token
should be rejected, not crash
the process.

```
/replay/protocol
```

```
READY
```

```
audit> parseProtocol("HTTP/")
```

IndexDefect

```
index out of bounds after unconditional dot skip
```

FILE: CODE PATH

The bug is a missing boundary check between two parse steps.

/src/vulnerable

READY

```
i.inc protocol.parseSaturatedNatural(result.major, i)
i.inc
i.inc protocol.parseSaturatedNatural(result.minor, i)
```

unconditional skip

/src/fixed

READY

```
i.inc protocol.parseSaturatedNatural(result.major, i)
if i < protocol.len: inc i
i.inc protocol.parseSaturatedNatural(result.minor, i)
```

guarded skip

Upstream reference: Nim PR #25793.

FILE: EVIDENCE CHAIN

A confirmed finding needs a chain, not a claim.

01 crash

fuzzer saves exact input

02 reduce

minimal payload is
isolated

03 reproduce

small Nim program triggers
the same defect

04 replay

fixed version rejects
safely

/audit/evidence

READY

audit> artifact: 0HTTP/

audit> minimal: HTTP/

audit> status: confirmed local denial-of-service crash

FILE: AI REVIEW

AI was useful before
confirmation, not after it.

- A

generate

possible risks and audit checklist
- B

triage

compare claims with source and harness scope
- C

validate

require a testcase before calling it a bug

/models/summary	READY
reported leads	23
likely worth testing	16
confirmed crashes	0
Best output: checklist material for a human security review.	

FILE: COMPARISON

CRITERION	FUZZING	AI REVIEW
confirmed crashes	1	0
evidence	crash input + replay	reasoning to validate
strength	runtime behavior	broad design coverage
weakness	harness scope	overstatement risk

Fuzzing answered “what happened.” AI helped decide “what to test next.”

FILE: OPERATING PRINCIPLE

/doctrine/evidence

READY

audit> model proposes

audit> harness tests

audit> developer validates

audit> fix is replayed

A finding is confirmed only when it can be reproduced.

FILE: REFERENCES

- [1] B. Grinstead, C. Holler, F. Braun, "Behind the Scenes Hardening Firefox with Claude Mythos Preview," Mozilla Hacks, May 2026.
- [2] LLVM Project, `libFuzzer` documentation.
- [3] Clang documentation, `AddressSanitizer`.
- [4] OWASP Foundation, `Fuzzing`.
- [5] `nim-lang/Nim`, PR 25793: fix DOS via malformed HTTP protocol.